benchmarked at each stage. The result is a 103.9 MB artifact, a fourfold compression from 411 MB.

Stage 4: Offline serving. An inference engine loads the quantised ONNX model and a local SentencePiece tokenizer and decodes greedily. A router resolves each request to a directly trained model where one exists, and otherwise chains two students through English. One engine serves a REST API, a command-line tool, an offline progressive web application and a browser client.

2.6.1 The Deciding Experiment

The project's founding hypothesis was H1: *preference distillation produces better compact students than sequence-level knowledge distillation*. It was tested at matched student size, on identical data, with evaluation on real human references held out from training, a design that is fair regardless of what each system trained on. Figure 2.1 gives the outcome.

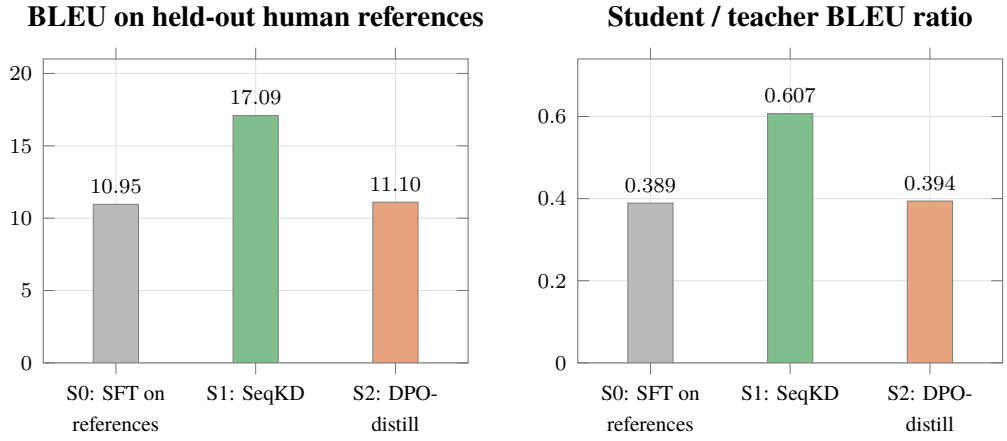


Figure 2.1: Head-to-head at 100,000 training sentences, Hindi→English, identical 52.6 M student, evaluated on held-out human references. SeqKD (S1) wins on both measures; DPO-distill (S2) is indistinguishable from the reference-trained baseline (S0)

H1 is refuted. Sequence-level distillation beats preference distillation by 6.0 BLEU, an improvement of about 54 %, and DPO applied on top of a reference-trained base adds 0.15 BLEU over that base, indistinguishable from noise. The replacement hypothesis, H1′, is supported by every subsequent result: *reference noise, not the training objective, is what caps a compact Indic student.* Teacher-distilled targets remove that noise, and the student then continues to improve with data: the ratio rises from 0.607 at 100,000 sentences to 0.764 at 250,000 and 0.806 at 500,000, whereas reference-trained students plateau near 0.66 regardless of data volume.

The DPO implementation is not defective, which is what makes the result informative. Its margin accuracy reaches 1.0, meaning it ranks preferred above dispreferred candidates perfectly; it consistently improves chrF by one to two points while costing about one BLEU point. The preference signal is being learned correctly; it simply is not the signal that a 52-million-parameter student most needs when its reference targets are noisy.

3.2.1 Use Case Diagram

Three actors interact with the system. The **End User** translates text. The **ML Engineer** runs the training pipeline. The **Edge Device** is a secondary actor that hosts the quantised model and executes inference.

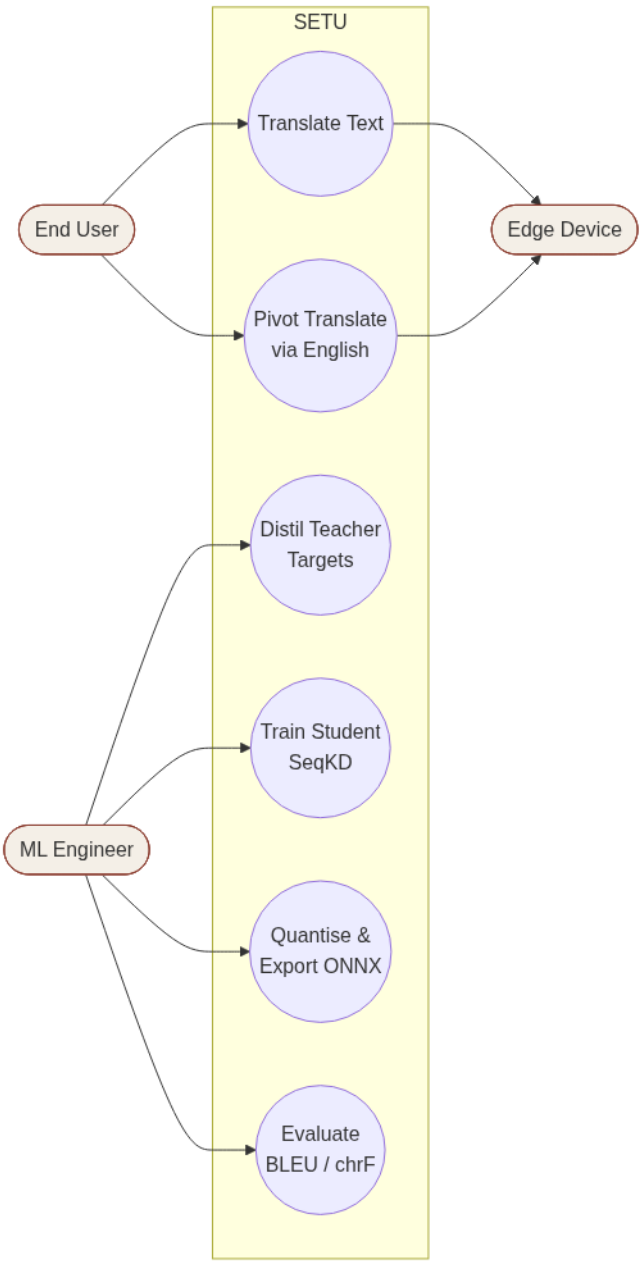


Figure 3.1: Use case diagram: End User, ML Engineer and Edge Device actors

Each use case is specified below by its actor, its precondition and its postcondition.

- **Translate Text (End User):** Precondition: a trained model exists for the requested direction. Postcondition: translated text is returned with measured latency.

3.2.2 Sequence Diagram

Figure 3.2 traces a single translation request. The significant branch is the pivot: when a directly trained model exists the engine performs one hop, and when it does not (every Indic-to-Indic pair) the engine performs two, passing the intermediate English string from the first student to the second. The response carries the summed latency of both hops and a `pivot` field so that the caller can tell the two cases apart.

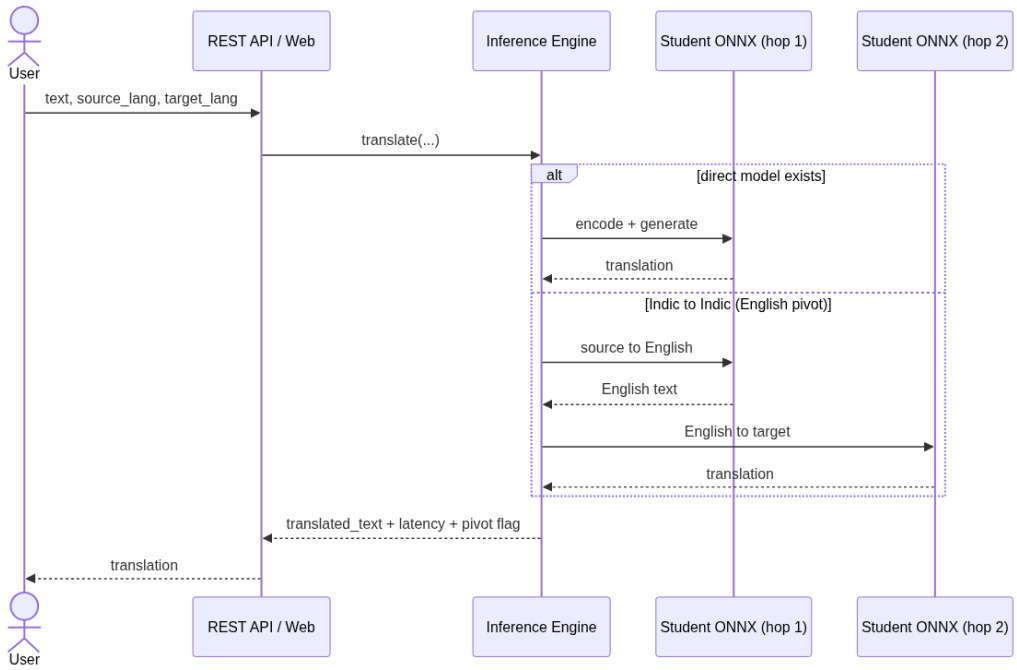


Figure 3.2: Sequence diagram: direct translation and the two-hop English pivot

3.2.3 Activity Diagram

Figure 3.3 shows the full training and deployment activity flow. Everything above the dividing line executes once per direction on a GPU; everything below executes on the user's device. The two decision points are the quality gate (does the student reach 80 % of teacher BLEU) and the routing decision at inference time.

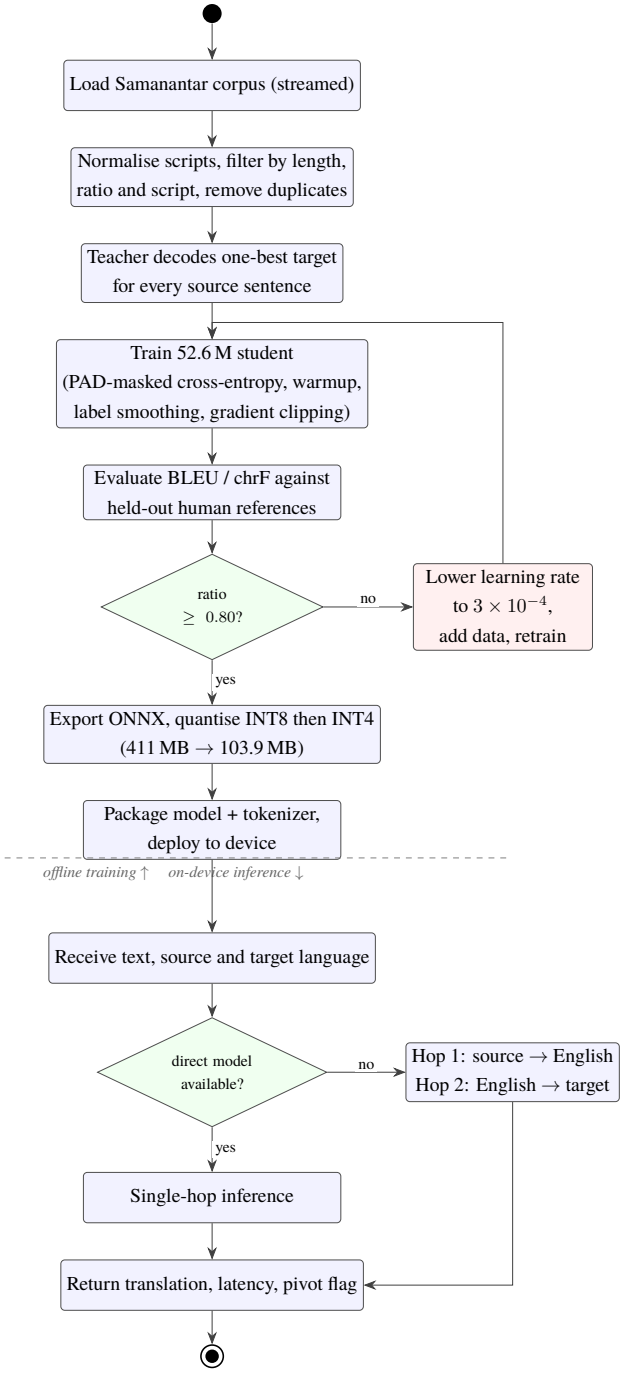


Figure 3.3: Activity diagram: offline training pipeline (above) and on-device inference (below)

3.2.4 State Chart Diagram

A model in SETU passes through a well-defined sequence of states. Figure 3.4 records them, including the two transitions that were exercised in practice: the collapse detected at evaluation and the recovery achieved by retraining at a lower learning rate on the same distilled corpus.

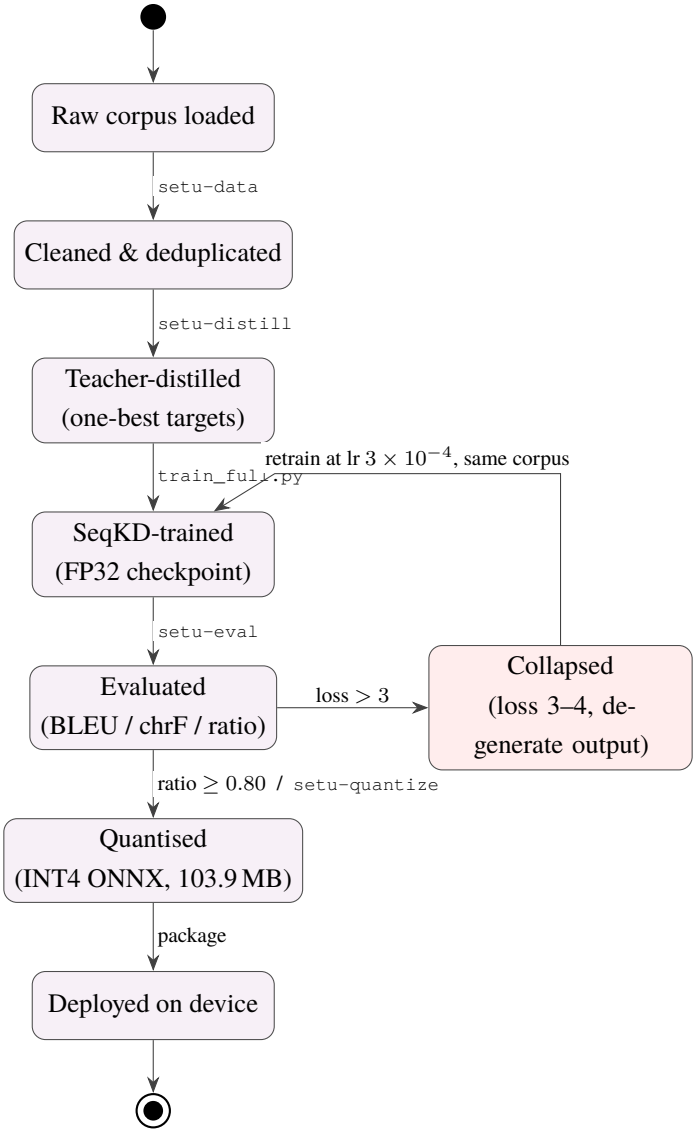


Figure 3.4: State chart diagram: model lifecycle from raw corpus to deployed artifact, including the observed collapse and recovery transitions

3.2.5 Data Flow Diagram

Figure 3.5 gives the level-1 data flow. The corpus is an external data source; the teacher is a transformation that produces the distilled data store; the trainer consumes that store and produces a student; the quantiser converts the student into a deployable artifact; the inference engine consumes the artifact and produces translations. The evaluator is a side branch that consumes both student and teacher outputs to compute the ratio that gates deployment.

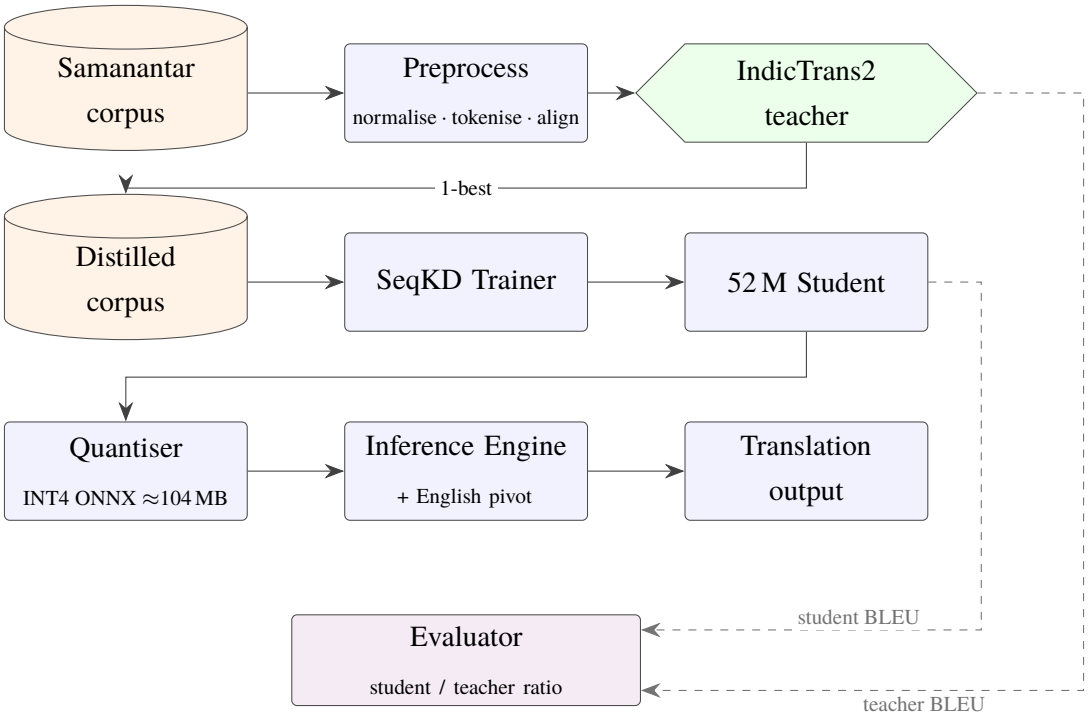


Figure 3.5: Level-1 data flow diagram: corpus to deployed translation

3.2.6 Entity-Relationship Model

SETU does not use a relational database (an offline edge deployment should not require one), but its persistent artifacts have a clear relational structure, recorded in Figure 3.6. Corpora are stored as JSONL, reports as JSON, models as ONNX with an accompanying SentencePiece tokenizer, and the language registry as YAML.

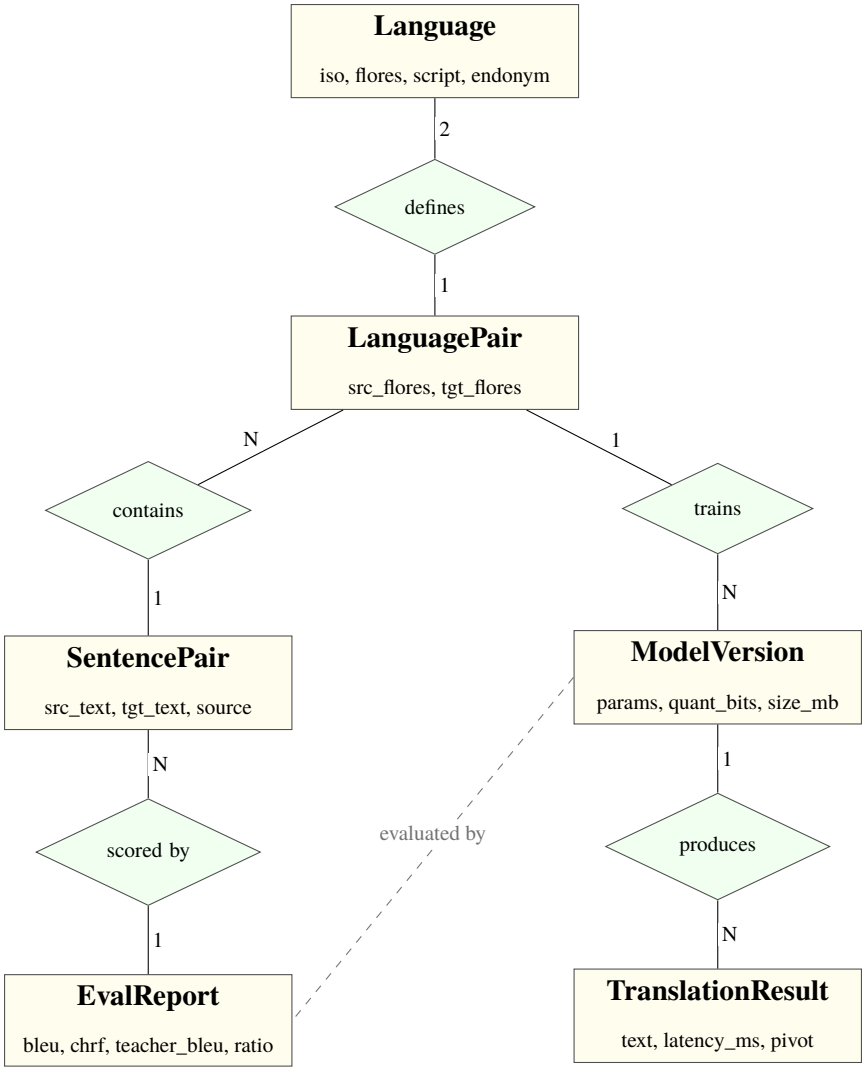


Figure 3.6: Entity-relationship model of SETU's persistent artifacts

3.2.7 Class Diagram

Figure 3.7 gives the principal classes. The design deliberately isolates every dependency on the teacher's implementation inside a single class, `IndicTrans2Teacher`. A unit test scans the source tree and fails the build if any other module imports `transformers`, `torch` or `IndicTransToolkit`. This is what allows the deployed artifact to contain no training dependencies at all.

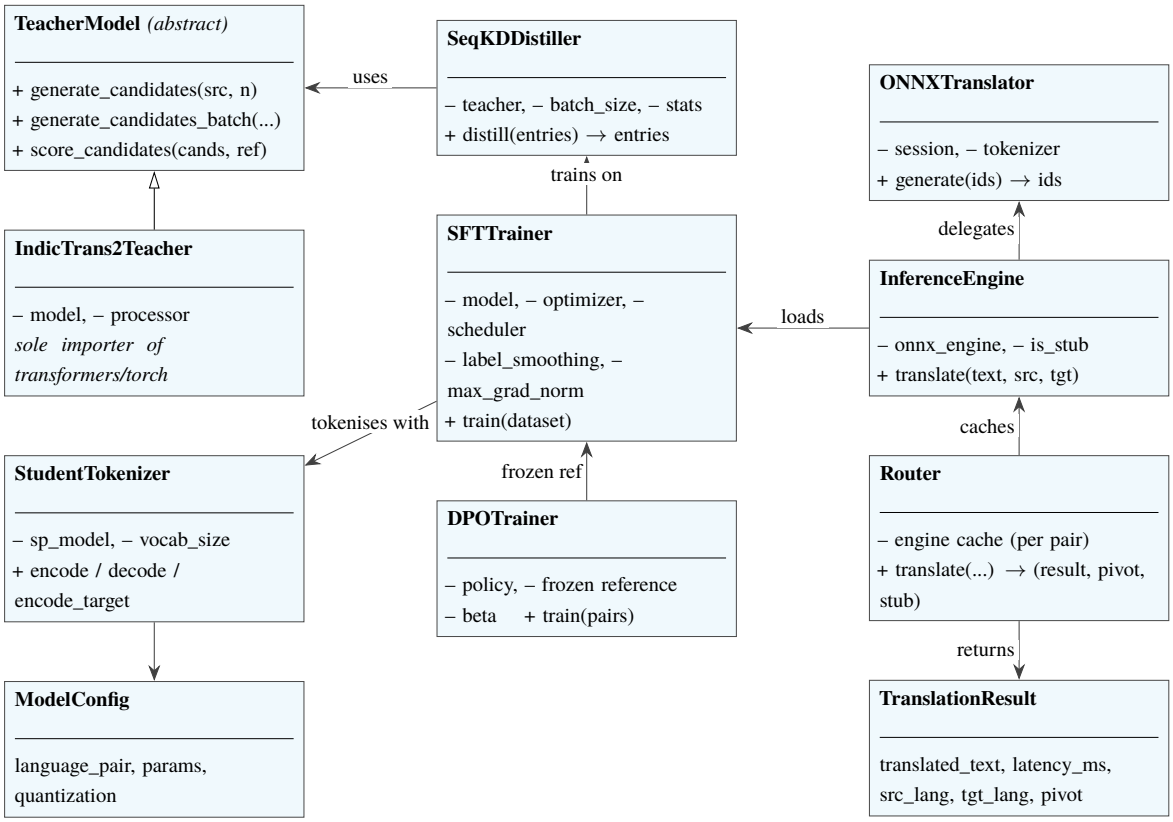


Figure 3.7: Class diagram: training classes (left) are used offline by the ML Engineer; inference classes (right) ship to the device

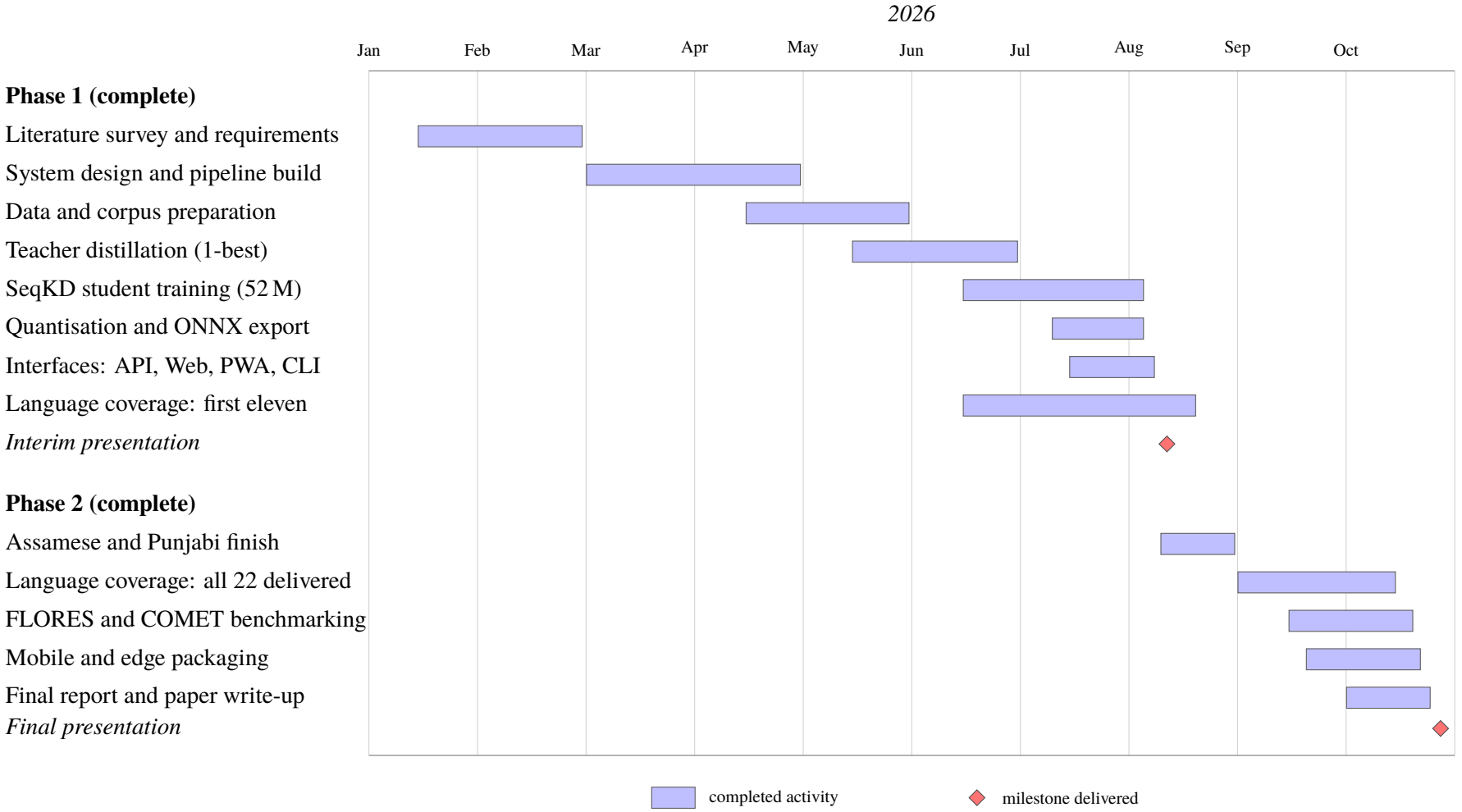


Figure 4.1: Gantt chart: both phases complete, all 22 scheduled languages delivered

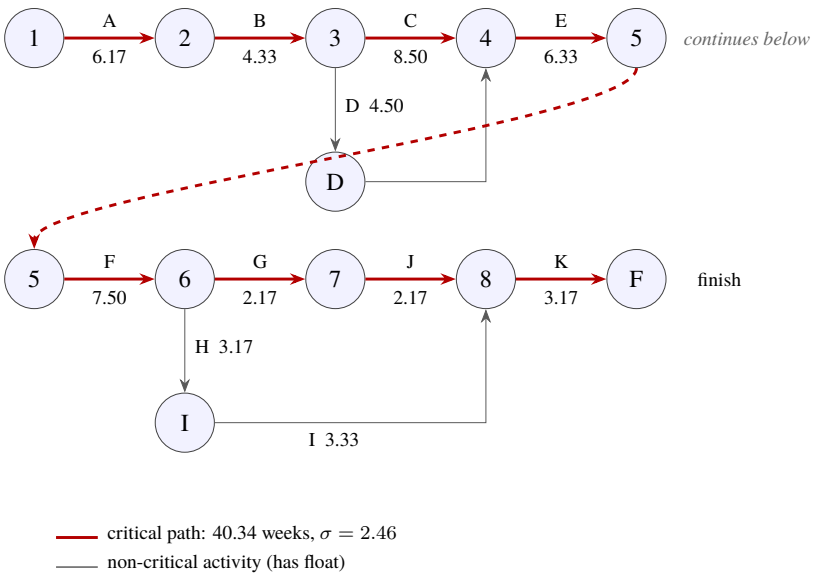


Figure 4.2: PERT network for Phase 1 with the critical path highlighted

4.1.4 Risk Management

The risks identified at planning time are recorded below, together with the mitigation applied and, where the risk materialised, what actually happened. Five of the nine anticipated risks occurred.

- **Gated access to the teacher checkpoints and the BPCC corpus.** Probability high, impact high. Fall back to ungated rotary checkpoints and the Samanantar corpus. **Occurred.** Mitigation applied; the cost is a lower quality ceiling, reported explicitly.
- **No GPU available locally.** Probability high, impact high. Use a cloud GPU (Kaggle, Colab) and later a dedicated A40 VM. **Occurred.** All final training ran on the A40 VM.
- **Student fails to reach the quality target.** Probability medium, impact high. Treat data volume, teacher strength and student width as independent levers; report shortfall honestly. **Partially occurred:** 14 of 22 directions pass.
- **Training instability or divergence.** Probability medium, impact high. Gradient clipping, learning-rate warmup, label smoothing; monitor training loss as a collapse indicator. **Occurred twice**; both recovered (Section 8.4).
- **Cloud session time limits kill long runs.** Probability high, impact medium. Resumable runners with per-stage completion markers; persist artifacts outside the session. **Occurred** on Kaggle's 12-hour cap; resumability absorbed it.
- **Quantised model exceeds the size budget.** Probability low, impact high. Progressive INT8 then INT4 with per-stage measurement. **Did not occur:** 103.9 MB against a 200 MB budget.

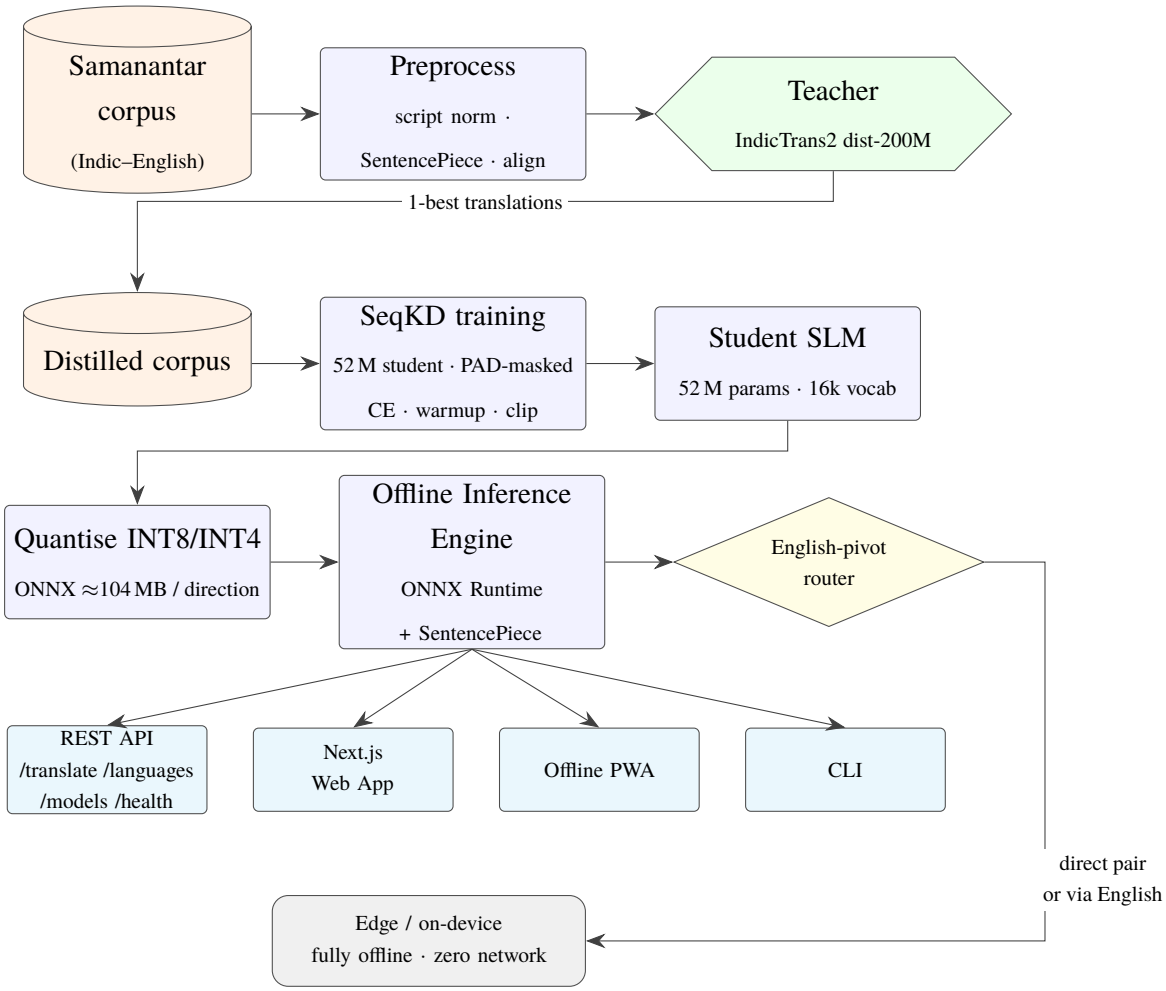


Figure 5.1: System architecture: SeqKD distillation and offline deployment pipeline

The architecture has four layers.

The data layer ingests parallel corpora, normalises and filters them, and stores two corpora per direction: the *processed* corpus, which retains human references and is used only for evaluation, and the *distilled* corpus, whose targets are the teacher's one-best translations and which is used for training. Keeping both is what allows evaluation to remain on real references regardless of what the student was trained on, the property that makes the comparison in Section 8.1 fair.

The model layer holds the teacher behind an abstract interface and the student behind a trainer. The teacher is used only offline, and only through three methods. The student is a Marian encoder–decoder trained from scratch.

The deployment layer converts a trained student into a device artifact: ONNX export, runtime validation, progressive INT8 and INT4 quantisation, and a per-stage benchmark. Its output is a self-contained directory.

The serving layer loads those directories on demand, caches one engine per direction, routes each request to a direct model or through the English pivot, and presents the

Loaders for FLORES-200 and IN22 are implemented for benchmark-grade numbers, along with COMET scoring and paired-bootstrap significance testing.

Two expansion mechanisms are implemented and tested. **All-in-One knowledge distillation** trains one student across several directions with temperature-sampled cross-lingual balancing and a coverage report; a sampling temperature above 1.0 flattens the distribution towards uniform and so protects low-resource directions from being swamped. **Elastic Weight Consolidation** adds a diagonal-Fisher penalty that anchors parameters important to previously learned languages, enabling a new language to be added without a full retrain. Its tests prove the penalty is zero at the snapshot point, grows with distance from it, and (measured rather than assumed) keeps Fisher-weighted drift on an old task below that of plain fine-tuning after a conflicting new task.

5.3 Interface Design

5.3.1 External Interfaces

Figure 5.2 shows how the four client surfaces reach the models. Every one of them resolves to the same inference engine, so behaviour cannot drift between them: the REST service is the only network-facing component, and the command-line tool bypasses it by linking the engine directly.

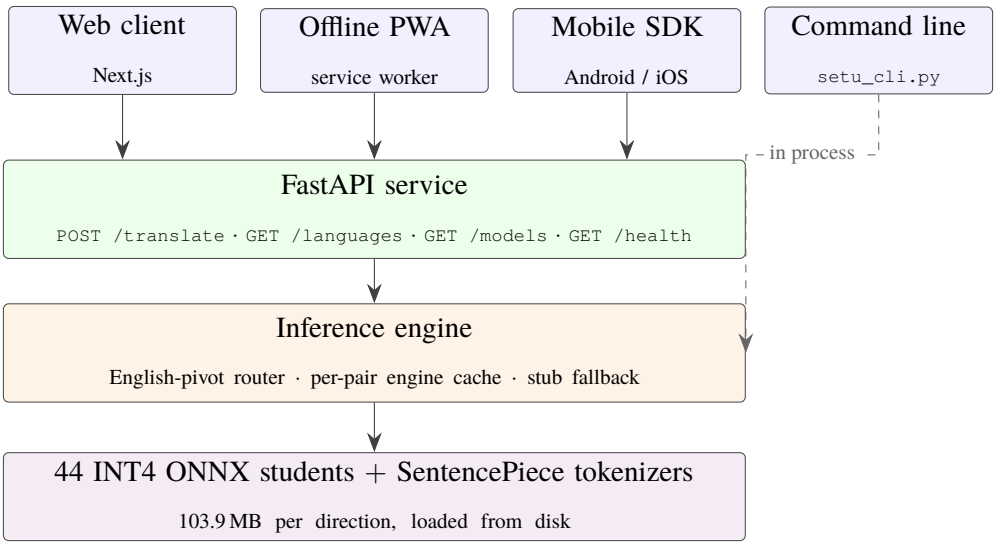


Figure 5.2: External interface design: four client surfaces over one shared inference engine

The interfaces themselves are specified below.

- **REST: translate.** Primary translation endpoint. `POST /translate` with body `{source_lang, target_lang, text}`; returns `{translated_text, latency_ms, pivot, stub}`. Example request: `{"source_lang":"hi",`

```python
class TeacherModel:
    """Abstract teacher. Only IndicTrans2Teacher may import
       transformers/torch."""

    def generate_candidates(self, src_text: str, src_lang: str,
                            tgt_lang: str, n: int = 4) -> list[str]:
        """Return n candidate translations, best first."""
        raise NotImplementedError

    def generate_candidates_batch(self, src_texts: list[str], src_lang: str,
                                  tgt_lang: str, n: int = 4) ->
                                      list[list[str]]:
        """Batched form -- the only method the SeqKD distiller needs."""
        raise NotImplementedError

    def score_candidates(self, candidates: list[str],
                         reference: str) -> list[float]:
        """Sentence-level chrF of each candidate against the reference."""
        raise NotImplementedError
```

Listing 6.2: The teacher interface (`src/setu/teacher/base.py`)

The wall is enforced mechanically. The test below walks the source tree and fails if any module other than the designated backend imports the teacher's dependencies. Without it the restriction would erode within weeks, and the deployed artifact would silently acquire a PyTorch dependency.

```python
FORBIDDEN = ("transformers", "torch", "IndicTransToolkit")
ALLOWED   = {"src/setu/teacher/indictrans2.py"}

def test_teacher_wall():
    """No module outside the backend may import the teacher's dependencies."""
    for path in Path("src/setu").rglob("*.py"):
        rel = path.as_posix()
        if rel in ALLOWED or "/training/" in rel or "/quantize/" in rel:
            continue                        # training and export legitimately
                use torch
        source = path.read_text()
        for name in FORBIDDEN:
            assert f"import {name}" not in source, f"{rel} imports {name}"
```

Listing 6.3: Enforcing the teacher wall (`tests/test_teacher.py`)

6.2.2 Corpus Cleaning

The cleaning filters encode domain knowledge about Indic text that a generic text pipeline would get wrong. Two details matter. Zero-width joiners and non-joiners must survive normalisation, because they distinguish valid conjuncts; a naive control-character strip corrupts correct text. And the script-fraction filter is what removes mined pairs in which one side is simply not in the claimed language.

```python
ZWJ, ZWNJ = "\u200d", "\u200c"   # zero-width joiner / non-joiner

def normalize(text: str) -> str:
    """NFC normalise and strip control characters, preserving ZWJ/ZWNJ."""
    text = unicodedata.normalize("NFC", text)
    return "".join(
        ch for ch in text
        if unicodedata.category(ch) != "Cc" or ch in (ZWJ, ZWNJ, "\n", "\t")
    )

def passes_length_ratio(src: str, tgt: str, max_ratio: float = 3.0) -> bool:
    """Reject misaligned pairs: mined corpora pair unrelated sentences."""
    if not src or not tgt:
        return False
    ratio = max(len(src), len(tgt)) / min(len(src), len(tgt))
    return ratio <= max_ratio

def passes_script_fraction(text: str, script: str, minimum: float = 0.5) ->
    bool:
    """At least `minimum` of the letters must belong to the expected
        script."""
    letters = [ch for ch in text if ch.isalpha()]
    if not letters:
        return False
    in_script = sum(1 for ch in letters if script_of(ch) == script)
    return in_script / len(letters) >= minimum
```

Listing 6.4: Corpus filters (`src/setu/corpus/filters.py`)

6.2.3 The SeqKD Distiller

The distiller is short, which is appropriate: sequence-level distillation is a simple idea, and the project's contribution is the evidence that it is the right one, not the code that implements it.

```python
class SeqKDDistiller:
```

```
    """Replace each entry's human target with the teacher's 1-best
        translation."""

    def __init__(self, teacher: TeacherModel, batch_size: int = 32):
        self.teacher = teacher
        self.batch_size = batch_size
        self.stats = {"entries": 0, "empty": 0}

    def distill(self, entries: list[CorpusEntry]) -> list[CorpusEntry]:
        out: list[CorpusEntry] = []
        for start in range(0, len(entries), self.batch_size):
            chunk = entries[start:start + self.batch_size]
            cands = self.teacher.generate_candidates_batch(
                [e.src_text for e in chunk],
                chunk[0].src_lang, chunk[0].tgt_lang, n=1,
            )
            for e, c in zip(chunk, cands):
                if c and c[0].strip():
                    tgt = c[0]                  # the teacher's target
                        replaces the human one
                else:
                    tgt = e.tgt_text            # safe fallback, never a
                        blank target
                    self.stats["empty"] += 1
                out.append(CorpusEntry(e.src_lang, e.tgt_lang,
                                       e.src_text, tgt, "seqkd"))
            self.stats["entries"] += len(chunk)
        return out
```

Listing 6.5: Sequence-level distillation (`src/setu/distill/distiller.py`)

The `source` field is set to `"seqkd"` so that a distilled corpus can never be mistaken for a reference corpus downstream. The evaluation code asserts that it is scoring against entries whose source is *not* `seqkd`, which is the mechanical guarantee behind the fairness of Section 8.1.

6.2.4 The Training Loop

This is the most-debugged code in the project. Every line commented below corresponds to a failure that was observed and diagnosed; the run history is in Section 8.3.

```
def train(self, dataset) -> dict:
    total_steps = len(dataloader) * self.epochs
    # Warmup was previously read from config and silently ignored; a constant
        LR
    # underfits a from-scratch transformer. Cap warmup at 10% of the run.
```

```
    warmup = min(self.config.get("warmup_steps", 0), max(1, total_steps //
        10))
    scheduler = get_linear_schedule_with_warmup(self.optimizer, warmup,
        total_steps)

    for epoch in range(self.epochs):
        for batch in dataloader:
            outputs = self.model(input_ids=batch["input_ids"],
                                 attention_mask=batch["attention_mask"],
                                 labels=batch["labels"])   # labels use -100
                                     at PAD
            if self.label_smoothing > 0:
                loss = F.cross_entropy(
                    outputs.logits.view(-1, outputs.logits.size(-1)),
                    batch["labels"].view(-1),
                    ignore_index=-100,                  # never score padding
                    label_smoothing=self.label_smoothing,
                )
            else:
                loss = outputs.loss
            loss.backward()
            # The single most important stabiliser: without clipping, a
                200k-sentence
            # run collapsed to constant output while 120k on the same code
                succeeded.
            torch.nn.utils.clip_grad_norm_(self.model.parameters(),
                self.max_grad_norm)
            self.optimizer.step()
            scheduler.step()
            self.optimizer.zero_grad()
```

Listing 6.6: Supervised training step with all four stabilisers (`src/setu/training/sft.py`)

The dataset builds the labels, and the single most consequential line in the entire training path is the padding mask:

```
def encode_target(self, text: str) -> list[int]:
    """Labels are [tokens, EOS] with NO leading BOS: HuggingFace already
        prepends
    the decoder start token, so adding one wastes a decode step and creates a
    train/generate mismatch that produced empty output in an early run."""
    ids = self.sp.encode(text)[: self.max_seq_len - 1]
    return ids + [self.eos_id]

def collate(self, batch: list[dict]) -> dict:
```

```
    labels = pad_sequence([b["labels"] for b in batch], batch_first=True,
                          padding_value=PAD_ID)
    # -100, not PAD: HuggingFace's loss uses ignore_index=-100. Padding with
        PAD
    # gave the model free credit for predicting PAD, producing a fake-low
        loss of
    # 0.274 and a decoder that ignored the source entirely.
    labels[labels == PAD_ID] = -100
    return {"input_ids": ..., "attention_mask": ..., "labels": labels}
```

Listing 6.7: Label construction (`src/setu/training/dataset.py`)

6.2.5 The Inference Router

The router is the whole of SETU's multilingual coverage story. It is also the one place where routing logic exists, which is what prevents the interfaces from diverging.

```
@lru_cache(maxsize=None)
def engine_for_pair(pair: str) -> InferenceEngine:
    """One cached engine per pair; a pair with no model on disk stays a
        stub."""
    return InferenceEngine(ModelConfig(language_pair=pair))

def translate(text, src_lang, tgt_lang) -> tuple[TranslationResult, str |
    None, bool]:
    """Returns (result, pivot, stub). Raises ValueError on unknown or equal
        codes."""
    src, tgt = resolve_language(src_lang), resolve_language(tgt_lang)
    if src["iso"] == tgt["iso"]:
        raise ValueError(f"Source and target language are both
            {src['iso']!r}")

    direct = f"{src['flores']}-{tgt['flores']}"
    if _available(direct):
        return engine_for_pair(direct).translate(text, src["iso"],
            tgt["iso"]), None, False

    # No direct model: chain source -> English -> target if both halves exist.
    eng = load_languages()["en"]["flores"]            # eng_Latn
    if src["iso"] != "en" and tgt["iso"] != "en":
        hop1, hop2 = f"{src['flores']}-{eng}", f"{eng}-{tgt['flores']}"
        if _available(hop1) and _available(hop2):
            r1 = engine_for_pair(hop1).translate(text, src["iso"], "en")
            r2 = engine_for_pair(hop2).translate(r1.translated_text, "en",
                tgt["iso"])
            return (TranslationResult(
```

```
                        translated_text=r2.translated_text,
                        src_lang=src["iso"], tgt_lang=tgt["iso"],
                        latency_ms=(r1.latency_ms or 0.0) + (r2.latency_ms or
                            0.0)),
                    "en", False)
    # Nothing can serve this pair: pass through, but say so.
    return engine_for_pair(direct).translate(text, src["iso"], tgt["iso"]),
        None, True
```

Listing 6.8: English-pivot routing (`src/setu/inference/router.py`)

6.2.6 Offline Verification

The offline guarantee is a claim about the absence of behaviour, which cannot be established by inspection alone. It is therefore tested by removing the capability entirely: all socket construction is patched to raise, and translation is then required to succeed.

```
def assert_offline():
    """Disable every socket so any network call raises instead of
        succeeding."""
    def _blocked(*args, **kwargs):
        raise RuntimeError("network access attempted during offline
            inference")
    socket.socket = _blocked
    socket.create_connection = _blocked

def test_onnx_engine_translates_offline(tmp_models):
    assert_offline()                          # no sockets from here on
    engine = InferenceEngine(ModelConfig(language_pair="hin_Deva-eng_Latn"))
    result = engine.translate("bhaarat ek vishaal desh hai.", "hi", "en")
    assert result.translated_text                        # non-empty
    assert not engine.is_stub                            # a real model
        answered
```

Listing 6.9: Proving offline operation (`tests/test_quantize.py`)

6.2.7 The REST Service

```
@app.post("/translate", response_model=TranslateResponse)
def translate(req: TranslateRequest) -> TranslateResponse:
    try:
        result, pivot, stub = router.translate(req.text, req.source_lang,
            req.target_lang)
    except ValueError as exc:                 # unknown code, or src == tgt
        raise HTTPException(status_code=400, detail=str(exc))
```

- **Sequence truncation at `max_seq_len`.** Sentences beyond the positional-embedding limit crashed training and evaluation. *Measured effect:* Fixed the crash; a regression test prevents recurrence.
- **Resumable stage markers.** A dropped SSH session discarded hours of work. *Measured effect:* Interrupted runs resume at the incomplete stage.
- **`expandable_segments` disabled.** The CUDA virtual-memory allocator is unsupported on a vGPU slice. *Measured effect:* Removed the immediate crash on first allocation.

6.2.8.1 The Quantisation Trade-off, Measured

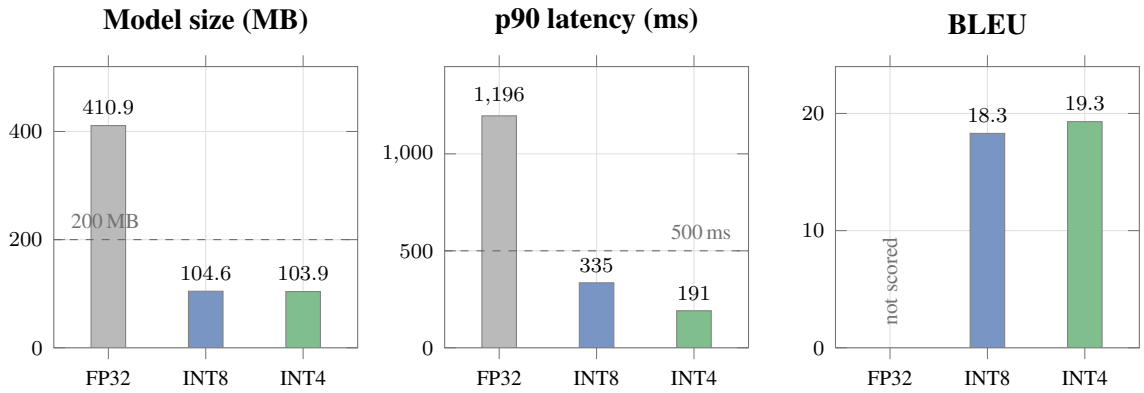


Figure 6.1: Effect of quantisation on size, latency and quality (Hindi→English student). INT4 is the shipped variant: it is the only precision that clears both the 200 MB and the 500 ms budget, at no measurable cost in BLEU

The arrows record an important intermediate finding. The first quantisation run produced latencies above the 500 ms budget not because quantisation was ineffective but because the undertrained model of that run decoded 128 junk tokens for every input. Once the training bugs were fixed and the model emitted sentences of normal length, the same quantised artifact measured 191 ms. The lesson is that a latency measurement on a broken model measures the breakage, not the runtime.

The quality figures also show that INT4 scored marginally *higher* than INT8 (19.3 against 18.3 BLEU). This is quantisation variance on a 500-sentence evaluation set rather than a real gain from lower precision, but it confirms that 4-bit quantisation costs nothing measurable here, which is why the INT4 variant is the one shipped.

- **Preference distillation (S2).** Supervised targets: human references; DPO: yes. The project's original proposal.
- **Distillation then DPO (S3).** Supervised targets: teacher one-best; DPO: yes. Does preference learning help on top of strong distillation?.

8.1.2 Result

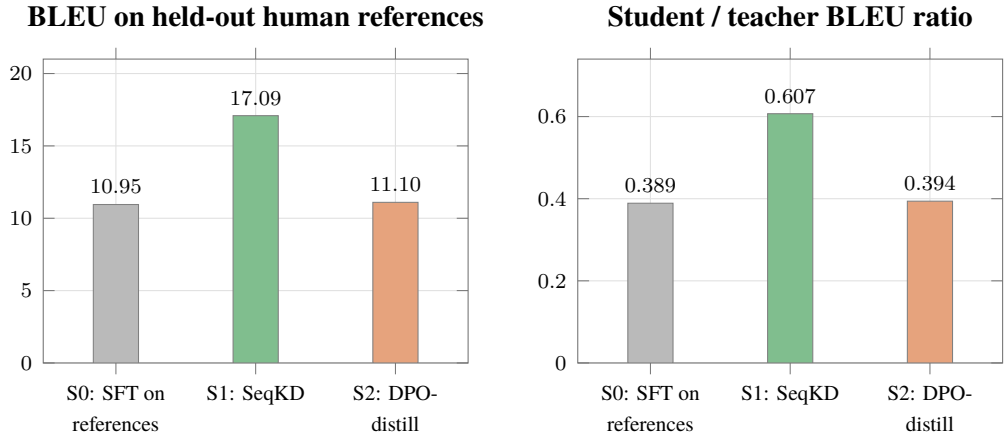


Figure 8.1: Head-to-head at 100,000 sentences, Hindi→English, matched student size, evaluated on human references (teacher BLEU 28.16). SeqKD (S1) gains $+6.14$ BLEU over the reference-trained baseline S0; DPO-distill (S2) gains $+0.15$

The hypothesis is refuted, and not marginally. Sequence-level distillation beats preference distillation by **5.99 BLEU**, an improvement of roughly **54 %**, at identical model size on identical data. Applying DPO on top of a reference-trained base adds 0.15 BLEU over that base, within the noise of a 500-sentence evaluation set.

8.1.3 Why the Hypothesis Failed

The most important thing about this result is that the DPO implementation is not defective. Three pieces of evidence establish that.

First, the **preference data is abundant and well separated**. A single run turned 8,000 corpus entries into 55,910 teacher candidates and 33,535 validated preference pairs, with a median chrF quality delta of 29.5 and a mean of 32.7. Every emitted pair passes a validation gate requiring the preferred candidate to score strictly above the dispreferred one.

Second, **DPO learns what it is asked to learn**. The margin accuracy, the fraction of pairs on which the trained policy ranks preferred above dispreferred, reaches 0.923 and later 1.0. The optimisation is succeeding completely at its stated objective.

Third, **DPO produces the effect its ranking metric predicts**. Because the preferences were ranked by chrF, DPO consistently improves chrF (from 37.85 to 40.33 in one

run, and from 45.40 to 45.94 in another) while costing approximately one BLEU point. The preference signal is being transferred faithfully; it simply is not the signal that most improves translation quality.

The explanation, and the project's replacement hypothesis H1′, is that **reference noise rather than the training objective is what caps a compact Indic student**. Samanantar is web-mined. Its sentence pairs vary in register, in alignment quality and in fidelity; roughly one pair in a hundred is discarded outright by the cleaning filters, and many more that survive are imperfect. A model with billions of parameters can average over that inconsistency. A model with 52 million cannot; it spends its limited capacity modelling the noise. The teacher's one-best output, by contrast, is internally consistent: it is drawn from a single distribution, in a single register, with alignment guaranteed by construction. Training on it is a denoising operation.

Two subsequent observations support H1′ directly. Reference-trained students plateau near a ratio of 0.66 and do not improve when data is increased from 200,000 to 250,000 sentences; the data lever saturates while the noise remains. Distilled students do not plateau; they continue improving to 0.806 at 500,000 sentences. If data volume were the binding constraint both curves would rise; if noise is the binding constraint only the denoised curve rises, which is what was observed.

8.2 Data Scaling of Sequence-Level Distillation

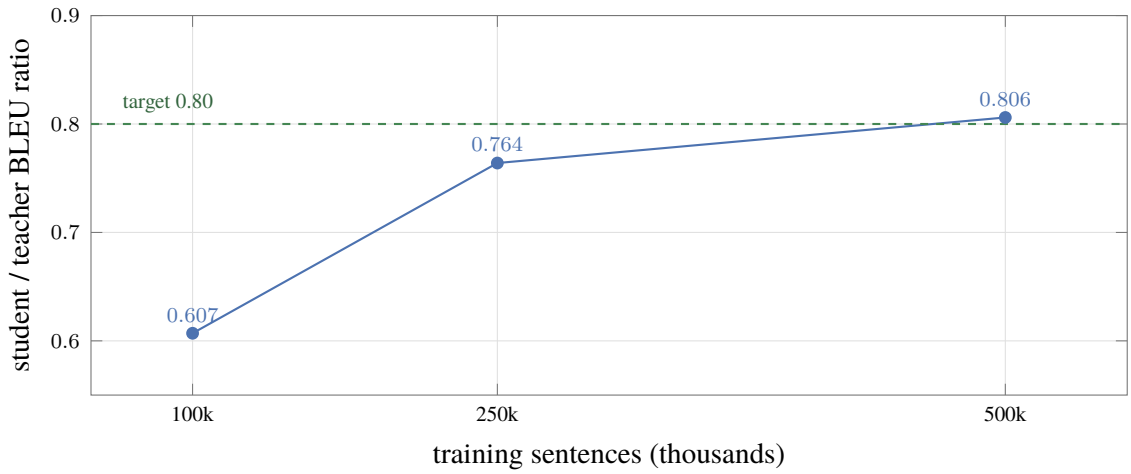


Figure 8.2: SeqKD data-scaling curve, Hindi→English. The ratio rises steeply to 250,000 sentences and then flattens as the student approaches its teacher's ceiling, crossing the 0.80 target at 500,000

The curve rises steeply from 100,000 to 250,000 sentences and then flattens, which is the expected shape as the student approaches its teacher's ceiling. The ungated distilled-200M

8.5 Test Reports

8.5.1 Full Scorecard: 22 Directions

Figure 8.3 gives every trained direction, ordered by student/teacher BLEU ratio. Eleven languages are covered, each in both directions.

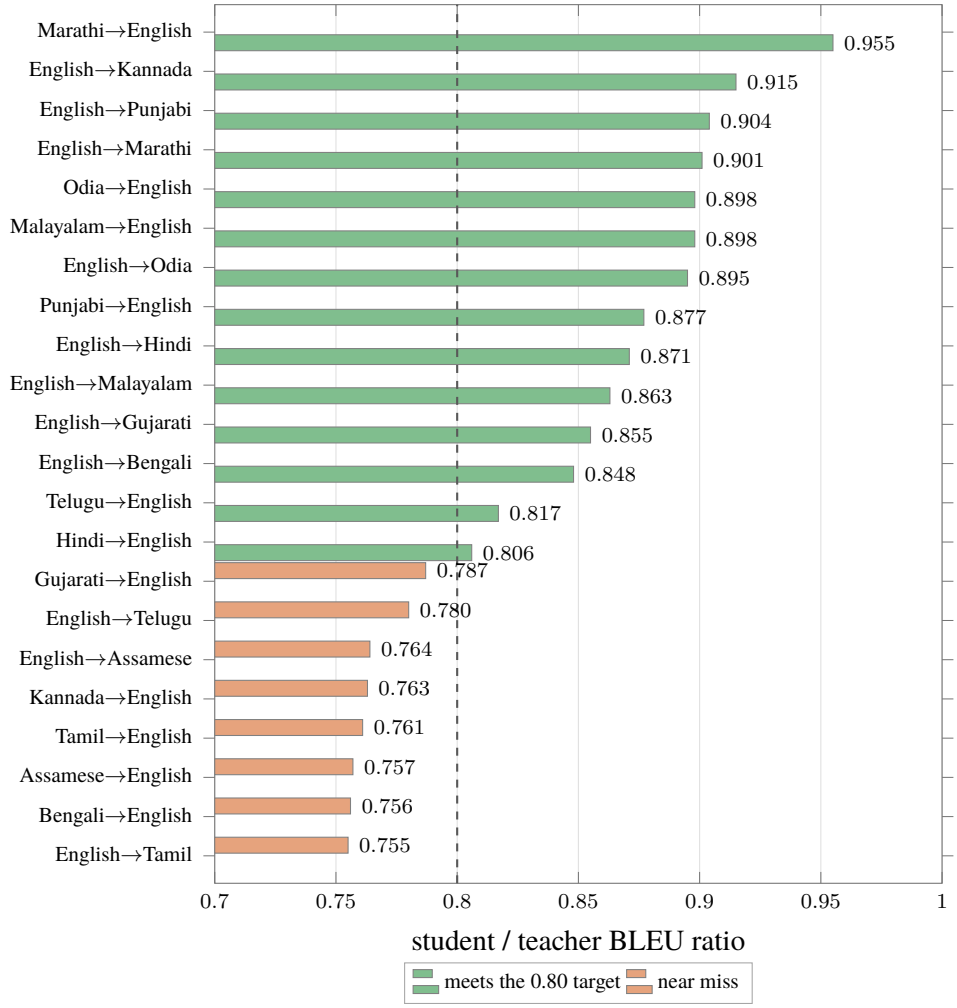


Figure 8.3: Student/teacher BLEU ratio for all 22 evaluated directions on the 500-sentence Samanantar development slice. Fourteen clear the 0.80 target; the dashed line marks the threshold

- **Directions trained:** 22 (11 languages, both ways).
- **Meeting the ≥ 0.80 target: 14 of 22 (64 %).**
- **Mean ratio: 0.838**.
- **Median ratio: 0.855**.
- **Best direction:** Marathi → English, 0.955.
- **Weakest direction:** English → Tamil, 0.755.

- **Collapsed directions remaining: 0** (both recovered).
- **Artifact size:** 103.9 MB, every direction.
- **Mean latency range:** 105–202 ms.
- **Offline operation:** Verified for every direction.

8.5.2 Per-Language Rollup and Quality Tiers

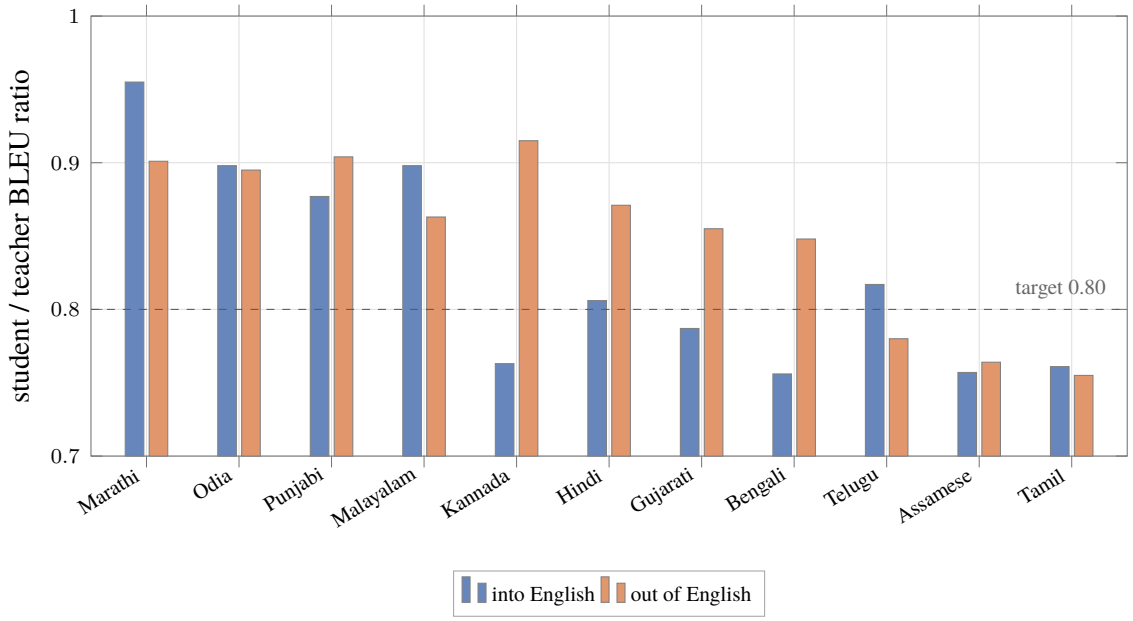


Figure 8.4: Per-language ratio in both directions for all eleven languages, ordered by their two-direction mean. The two directions track closely, so the method is not biased towards translating into English

Three observations follow.

Direction symmetry holds. Across all eleven languages, English→Indic averages 0.850 and Indic→English averages 0.825. The two directions are close, which suggests the method is not biased towards translating into English, a common asymmetry in multilingual systems.

The near-misses are under-converged, not unstable. All eight near-miss directions have supervised losses of 2.00 to 2.32, at the upper end of the healthy band of 1.5 to 2.3, whereas the passing directions cluster between 1.46 and 1.95. The pattern points to insufficient training rather than instability, which means the cheap remedies (beam search at evaluation, more epochs, more distilled data) are the appropriate ones.

BLEU understates agglutinative languages. English→Malayalam *passes* the target at 0.863 while scoring only 5.2 BLEU, because its teacher scores just 6.1. Malayalam's agglutinative morphology packs into single orthographic words what English expresses in

several, so n-gram overlap is structurally low even when the translation is correct. This is exactly why the project gates on the *ratio* rather than absolute BLEU, and why COMET scoring is recommended for Malayalam and Tamil: several near-misses may already pass under a metric that is not overlap-based.

8.5.3 Deployment Metrics: Size and Latency

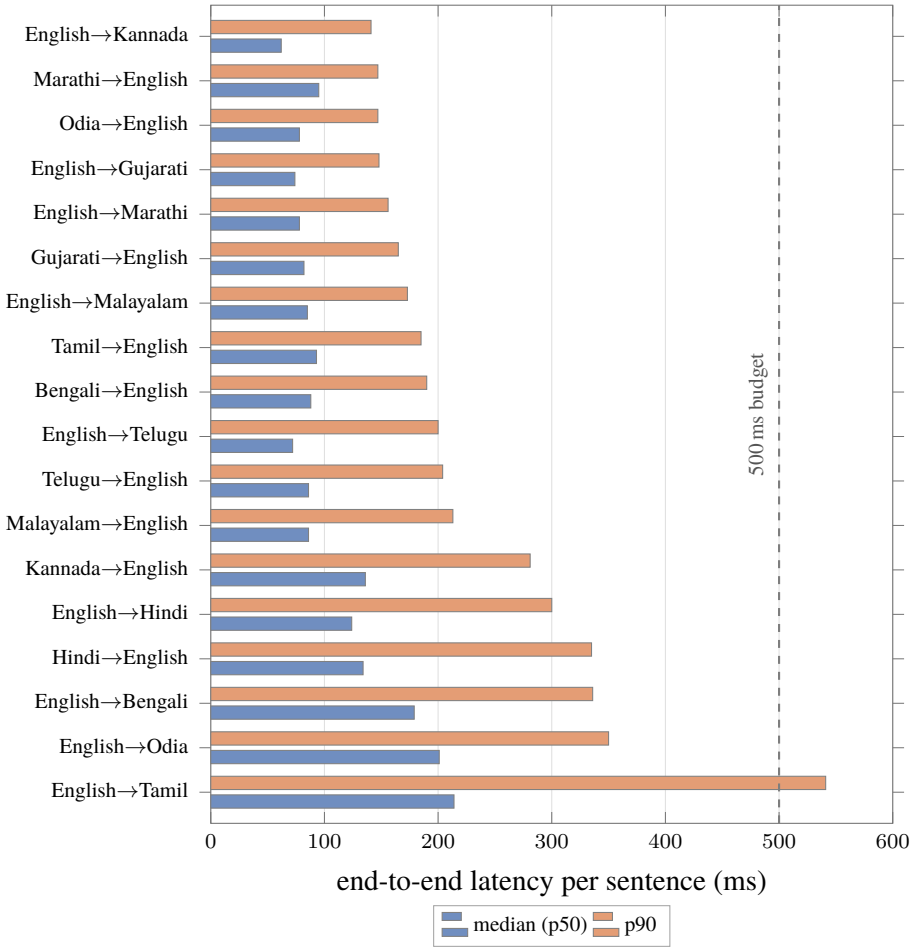


Figure 8.5: Median and p90 end-to-end latency for the 18 deployed directions (INT4 ONNX on the VM host CPU), ordered by p90. Every median sits well inside the 500 ms budget; only English→Tamil crosses it at p90

Every model compresses by a factor of 4.0, from 410.9 MB to 103.9 MB, which is 48 % below the 200 MB budget. Median latency is comfortably inside the 500 ms budget for every direction. The single exception is English→Tamil, whose p90 of 541 ms exceeds the budget; its median of 214 ms does not. This was measured on a shared VM host under contention, and Tamil's longer output sequences make it the most sensitive direction to host load. It is recorded as a near-miss rather than discounted.